

Angular Moderno

17 - Consumo de API de Produtos

2026

SENAI - CRTI

17 — Consumo de API de Produtos

Listagem dinâmica e integração com UI no Angular moderno
(Standalone + Signals)

i **Objetivo da aula:** Evoluir a implementação da aula anterior, saindo de uma abordagem didática (HTTP no componente) para uma abordagem profissional (Service + separação de responsabilidades)



Contexto da Aula Anterior

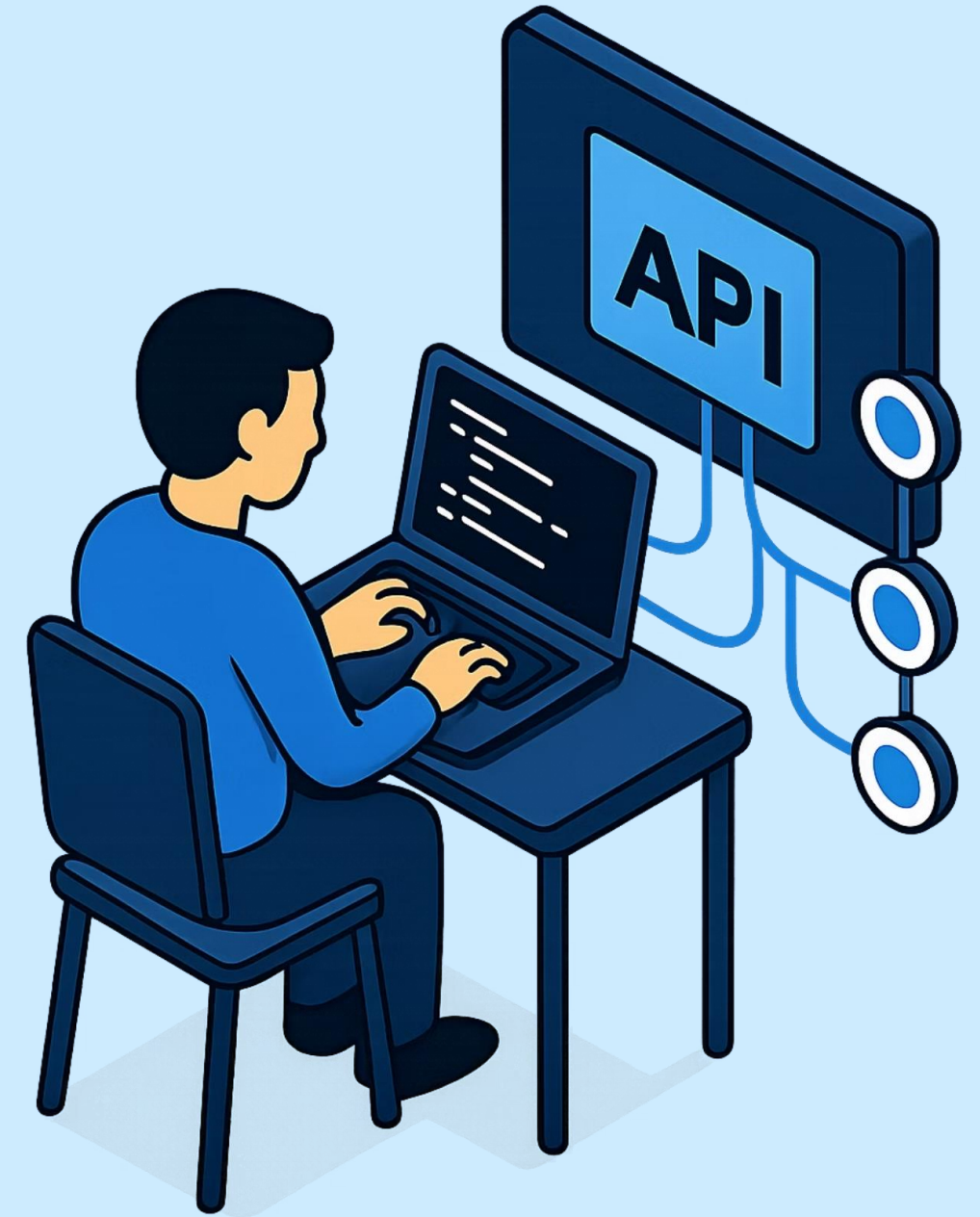
O que foi feito na aula anterior

- Implementamos `HttpClient` diretamente no componente
- Buscamos dados da API
- Atualizamos a UI com Signals

Problema atual

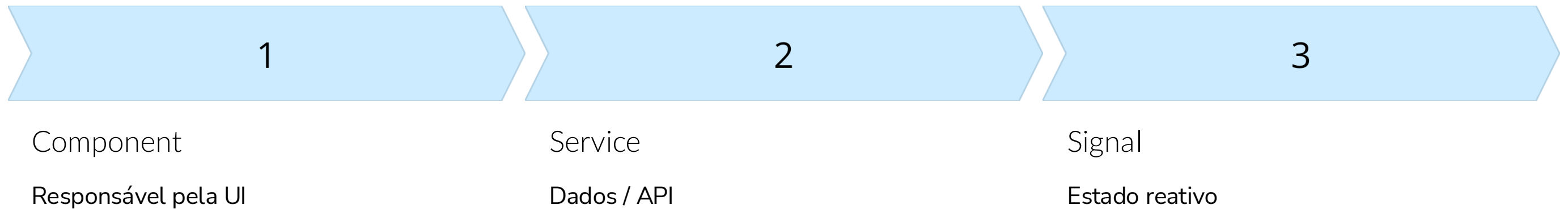
- O componente está fazendo muitas responsabilidades
- Código não escalável
- Mistura de lógica de UI com acesso a dados

⚠ Nesta aula: vamos corrigir isso com arquitetura profissional



Conceito Central da Aula

A regra de arquitetura que guia toda a aula:



Antes (Aula Anterior)

Component → API

Agora

Component → Service → API

✅ Essa é a mudança mais importante da aula, porém, ela **NÃO** altera o comportamento visual da aplicação

Etapa 0: Parar o Servidor

Antes de alterar qualquer arquivo, é necessário parar o servidor de desenvolvimento.

❌ No terminal: pressione **CTRL + C** para encerrar o processo

Evitar erros

Previne erros de compilação durante alterações estruturais

Segurança

Garante que alterações estruturais sejam aplicadas com segurança

Boa prática

Padrão recomendado no desenvolvimento Angular

Mesmo em alterações simples, vamos manter esse padrão profissional



Etapa 1: Criar o Service (local correto)

Arquivo a criar

```
features/produtos/produtos.service.ts
```

O service **NÃO** fica em `core` — ele pertence à feature `produtos`

Não usar core 

Service pertence ao domínio de produtos, não é global

Justificativa

- Não é um serviço global da aplicação
- Está ligado ao domínio de produtos
- Segue o princípio de coesão por feature



Colocar em features

Salvar em `features/produtos/produtos.service.ts`

Código do Service

PRODUTOS.SERVICE.TS

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

type ProdutoApi = {
  title: string;
  price: number;
};

type Produto = {
  nome: string;
  preco: number;
};

@Injectable({ providedIn: 'root' })
export class ProdutosService {
  private http = inject(HttpClient);

  private API = 'https://fakestoreapi.com/products';

  buscarProdutos() {
    return this.http.get<ProdutoApi[]>(this.API);
  }

  transformarProdutos(dados: ProdutoApi[]): Produto[] {
    return dados.map((p) => ({
      nome: p.title,
      preco: p.price,
    }));
  }
}
```

Explicação do Service

`inject(HttpClient)`

Padrão moderno do Angular para injeção de dependências — substitui o constructor injection tradicional

`providedIn: 'root'`

Torna o service disponível globalmente na aplicação sem necessidade de declaração em módulos

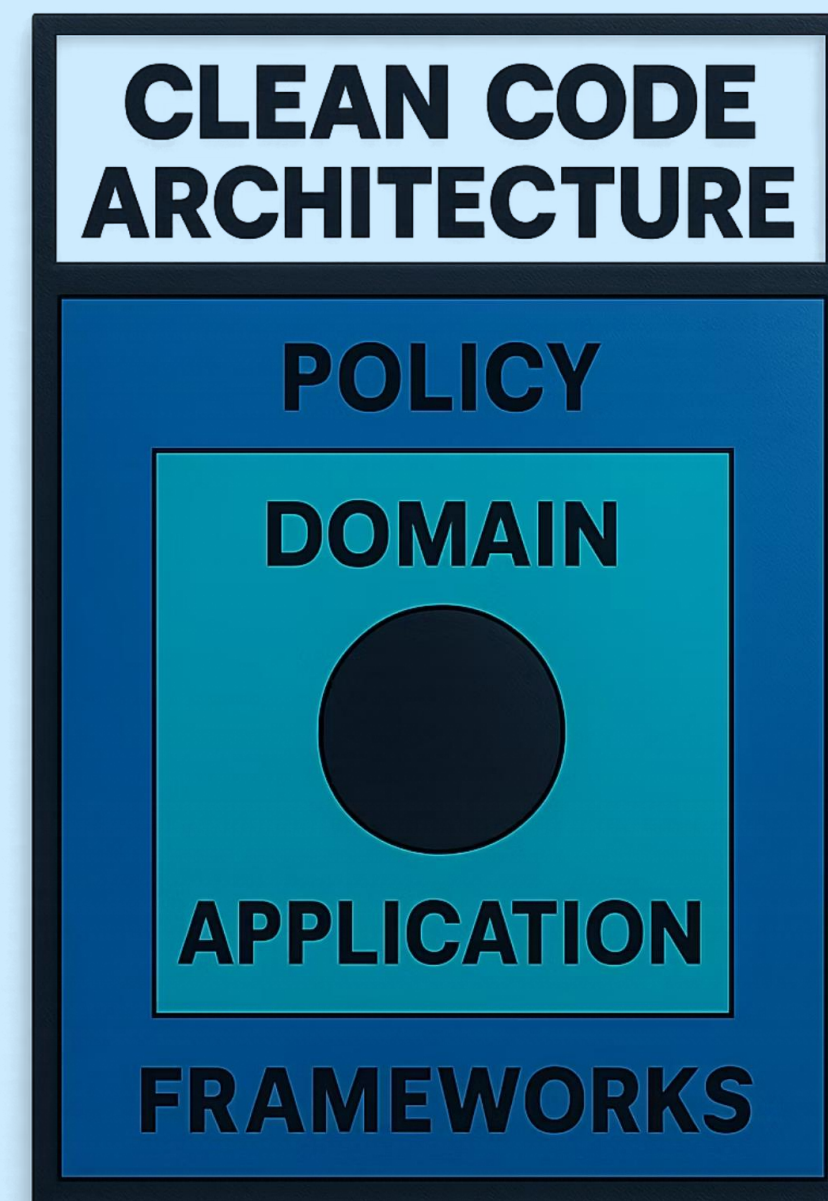
`buscarProdutos()`

Responsável exclusivamente pela comunicação com a API — retorna o Observable bruto

`transformarProdutos()`

Responsável pela adaptação dos dados da API para o formato interno da aplicação

- ✅ **Benefício:** Código reutilizável, organizado e com responsabilidades bem definidas. O componente não precisa saber de onde os dados vêm.



Etapa 2: Ajustar Import no Componente

LISTA-PRODUTOS.TS

✗ Remover este import

```
import { HttpClient } from '@angular/common/http';
```

Por que remover?

O componente **não deve mais acessar HTTP diretamente**. Essa responsabilidade agora pertence exclusivamente ao `ProdutosService`.

- Componente deve focar apenas na UI
- Acesso a dados é papel do Service
- Reduz acoplamento desnecessário

Essa remoção NÃO quebra a aplicação, pois o acesso agora está no Service.

Etapa 3: Importar Service e inject

LISTA-PRODUTOS.TS

Adicionar import do Service:

```
import { ProdutosService } from '../produtos.service';
```

Ajustar import do Angular (adicionar `inject`):

```
import { Component, signal, computed, effect, inject } from '@angular/core';
```

⊗ **ATENÇÃO IMPORTANTE:** Sem o import do `inject`, ocorre o erro: *"Não é possível encontrar o nome 'inject'"*. Este é um dos erros mais comuns nesta etapa!

Etapa 3: Inserir o inject no local correto

LISTA-PRODUTOS.TS

Dentro da classe **ListaProdutos**, adicionar:

```
private produtosService = inject(ProdutosService);
```

✓ Posição correta

Junto com os atributos da classe (signals, computed, etc.) — no corpo da classe, fora de qualquer método

✗ NÃO colocar

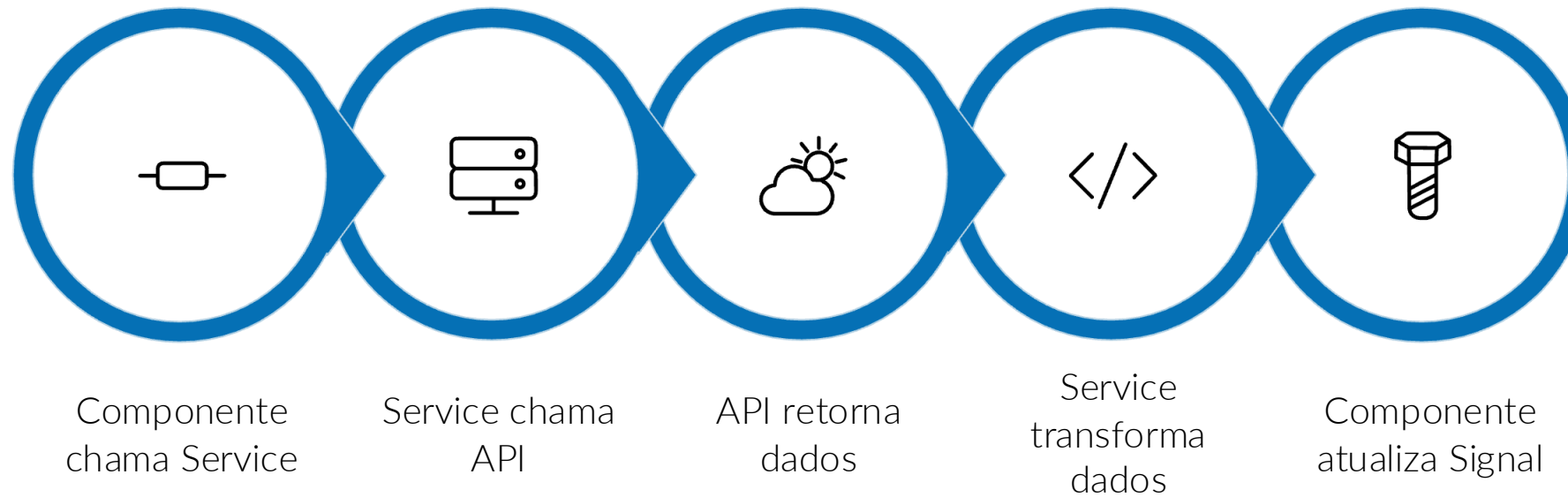
- Dentro do **constructor()**
- Fora da classe
- Dentro de métodos

Etapa 4: Substituir método *carregarProdutos()* por este:

```
carregarProdutos() {  
  this.carregando.set(true);  
  
  this.produtosService.buscarProdutos().subscribe({  
    next: (dados) => {  
      const produtos = this.produtosService.transformarProdutos(dados);  
      this.produtos.set(produtos);  
      this.carregando.set(false);  
    },  
    error: (erro) => {  
      console.error('Erro ao carregar produtos:', erro);  
      this.carregando.set(false);  
    },  
  });  
}
```

 Observe que o componente apenas orquestra — não conhece detalhes da API.

Explicação do Novo Fluxo



O fluxo completo agora passa pelo Service como intermediário entre o componente e a API, garantindo separação clara de responsabilidades.



Separação de responsabilidades

Cada camada faz apenas o que é seu papel



Código limpo

Componente enxuto, focado na UI



Fácil manutenção

Alterações isoladas por camada



Escalável

Pronto para crescer com o projeto

Etapa 5: Ajustar (remover) os argumentos do Constructor

LISTA-PRODUTOS.TS

```
constructor() {  
  this.carregarProdutos();  
  
  effect(() => {  
    console.log('Lista de produtos alterada:', this.produtos());  
  });  
  
  effect(() => {  
    console.log('Valor total atualizado:', this.valorTotal());  
  });  
  
  effect(() => {  
    if (typeof document !== 'undefined') {  
      document.title = `(${this.totalProdutos()}) Minha Loja`;  
    }  
  });  
}
```

 O constructor chama `carregarProdutos()` na inicialização e registra os `effect()` para reagir automaticamente às mudanças de estado via Signals.

O Template

LISTA-PRODUTOS.HTML

1

Bloco de Loading

Uso de `@if` para exibir indicador de carregamento enquanto os dados são buscados na API

2

Listagem de Produtos

Uso de `@for` para iterar sobre os produtos e renderizar cada item dinamicamente

3

Separação Visual

Uso de `<hr>` para separar seções visualmente na interface

4

Reatividade Automática

A UI reage automaticamente aos Signals — sem necessidade de chamadas manuais de detecção de mudanças

Etapa 6: Template completo

LISTA-PRODUTOS.HTML

```
<!-- Seção de Produtos -->
<section>
  <h2>Produtos:</h2>

  @if (carregando()) {
    <p>Carregando produtos...</p>
  }

  @if (!carregando()) {
    @for (item of produtos(); track item.nome) {
      <app-produto
        [nome]="item.nome"
        [preco]="item.preco"
        (produtoSelecionado)="exibirProduto($event)"
      ></app-produto>
      <button (click)="adicionarAoCarrinho(item)">Adicionar ao Carrinho</button>
    }
  }
  <h2>
    Produto selecionado:
    @if (produtoSelecionado()) {
      <strong style="color: green">{{ produtoSelecionado() }}</strong>
    } @else {
      <span style="color: gray">Nenhum</span>
    }
  </h2>
```

```
<p><button (click)="adicionarProduto()">Adicionar Produto</button></p>

<p><button (click)="substituirProdutos()">Substituir Lista</button></p>

<h2>Total de produtos: {{ totalProdutos() }}</h2>

<h3>Valor total: R$ {{ valorTotal() }}</h3>
</section>

<!-- Seção do Carrinho -->
<section>
  <hr />

  <h2>Carrinho:</h2>

  @for (item of carrinho(); track $index) {
    <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>
  }

  <p><strong>Itens no carrinho:</strong> {{ quantidadeCarrinho() }}</p>
  <p><strong>Total do Carrinho:</strong> R$ {{ totalCarrinho().toFixed(2) }}</p>
</section>
```

Manter Funcionalidades Existentes

Não remover os seguintes elementos do componente:

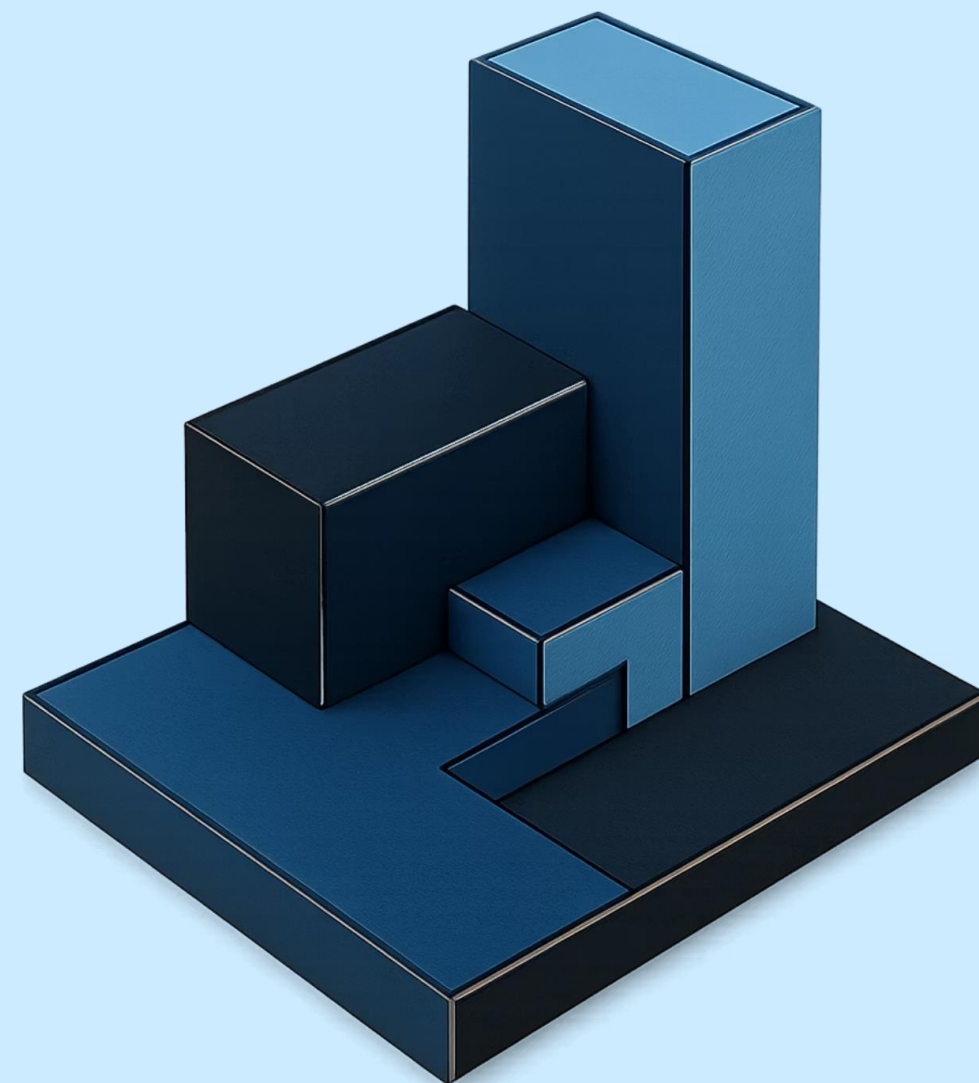
Manter no componente

- `produtoSelecionado`
- `carrinho`
- `computed (totais)`

Justificativa

- São **estados de UI** — não fazem parte da API
- Representam interações do usuário com a interface
- Continuam sendo responsabilidade do componente
- Não devem ser movidos para o Service

📄 A regra é simples: se é estado de UI (seleção, carrinho, totais calculados), fica no componente. Se é acesso a dados externos, vai para o Service.



Testes da Aplicação

Após rodar o servidor:

```
ng serve
```

1

Loading aparece

O indicador de carregamento deve ser exibido enquanto a requisição está em andamento

2

Produtos vêm da API

Os produtos devem ser carregados corretamente da `fakestoreapi.com`

3

UI atualiza automaticamente

A interface deve reagir aos Signals sem necessidade de intervenção manual

4

Console sem erros

Nenhum erro deve aparecer no console do navegador após a implementação

O que Mudou de Verdade

Visualmente

Quase nada mudou para o usuário final. A aplicação continua exibindo os mesmos produtos, com o mesmo visual e as mesmas interações.

Arquiteturalmente

Mudou tudo. A estrutura interna do código foi completamente reorganizada seguindo princípios profissionais de desenvolvimento.

Antes

Component → API

Acoplamento direto, difícil de testar e manter

Agora

Component → Service → API

Camadas bem definidas, testável e escalável

Conclusão da Aula

O que foi aprendido nesta aula:

01

Separação de responsabilidades

Component cuida da UI, Service cuida dos dados — princípio fundamental da arquitetura Angular

02

Uso de Service no Angular moderno

Criação e organização de services por domínio/feature, com `providedIn: 'root'`

03

Uso correto de `inject()`

Padrão moderno de injeção de dependências, substituindo o constructor injection

04

Integração limpa entre API, Signals e UI

Fluxo de dados organizado: API → Service → Signal → UI reativa

 **Próximas aulas:** Interceptors · Tratamento de erros avançado · Arquitetura profissional completa



Dúvidas

?

